Finite Automata and Regular Expression

Zhengfeng Ji, *Department of Computer Science and Technology, Tsinghua University*

1 State Elimination Method

1.1 Converting DFA's to RE by eliminating states

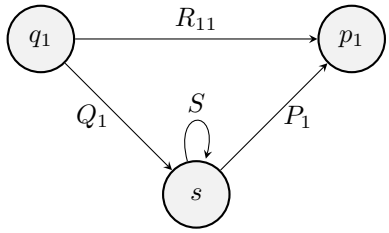
- Issues for the above method:
  1. About n^3 intermediate expressions.
  2. The length of the expression could grow as 4^n for an n-state automaton.
- Each $R_{kk}^{(k-1)}$ was written in about n^2 expressions.

1.2 Idea

- The approach to constructing regular expressions we shall now learn involves eliminating states. When we eliminate a state s all the paths that went through s no longer exist in the automaton.
- Automata with RE as labels!

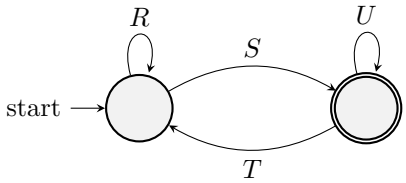
1.3 The eliminating method

- Eliminate a state s

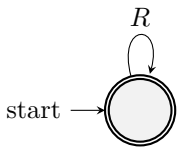


$R_{11} + Q_1 S^* P_1$

- After the elimination, we will get a two-state or one-state automaton:



$(R + SU^*T)^*SU^*$



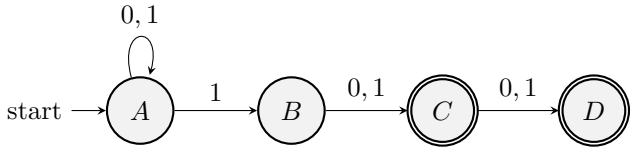
R^*

- State elimination procedure:

> 1. For ***each accepting state*** q, eliminate all but q and q_0.
> 2. Take the union.

1.4 Example (Example 3.6 of the textbook)

- An automaton accepts all strings whose second or third position from the end has a 1.



Step 1. Use RE as labels.

Step 2. Eliminate B. A to C is empty, $1\emptyset^*(0+1)$

- Then we branch (for all accepting states)
  1. First, consider those strings accepted at D. We eliminate C.
  2. Second, consider those strings accepted at C. We eliminate D. There is no arc added.
- Final answer

$(0+1)^*1(0+1) + (0+1)^*1(0+1)(0+1)$

2 From RE's to Automata

> The language $L(R)$ of RE R is also $L(E)$ for some ϵ-NFA E.

- ***Thompson method***

Proof. Structural induction on R.

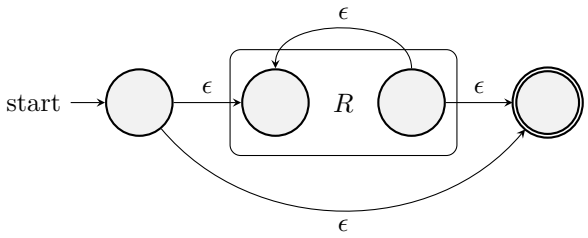
Simplification: E satisfies the following three extra conditions:

1. Exactly one accepting state
2. No arcs into the initial state
3. No arcs out of the accepting state

(Note: 2 and 3 are used to avoid extra discussions in the R^* construction)

Basis: Automata for accepting ϵ, $\emptyset$, a satisfying the ***three extra conditions***.

Induction: $R+S$, RS, R^*.



See Figure 3.17 of the textbook for the full construction.

□

Example. Convert the regular expression $1(0+1)^*$ to an ϵ-NFA.

3 Algebraic Laws for RE

3.1 What is an algebraic law

- In mathematics

  $1+2=2+1$ vs $x+y=y+x$

  Commutativity: The equality holds for all values of the variables x and y

  Algebra: addition and multiplication

- Algebraic laws for RE
  - Language L as a variable.
  - Union as the addition.
  - Concatenation as multiplication.

3.2 Associativity and commutativity

$(L+M)+N=L+(M+N)$

$L+M=M+L$

$(LM)N=L(MN)$

But not $LM=ML$.

3.3 Identities and annihilators

$\emptyset+L=L$, identity for addition

$\epsilon L=L\epsilon=L$, identity for concatenation

$\emptyset L=\emptyset$, annihilator for concatenation

3.4 Distributive laws

Let L, M, N be regular expressions. It holds that $L(M+N)=LM+LN$.

How to prove it? By definition and set equality.

Proof. First, we prove $L(M+N)\subseteq LM+LN$. For all $w\in L(M+N)$, there is a partition $w=w_1w_2$ such that $w_1\in L$ and $w_2\in M+N$. Case 1, $w_2\in M$, $w\in LM$. Hence $w\in LM+LN$. Case 2, $w_2\in N$, $w\in LN$. Hence $w\in LM+LN$.

Second, we prove $LM+LN\subseteq L(M+N)$.

□

3.5 Idempotent law for union

$L+L=L$

3.6 Laws involving closures

$(L^*)^*=L^*$, any concatenation of concatenations is a concatenation

$\emptyset^*=\epsilon$,

$\epsilon^*=\epsilon$

$L^+=LL^*$

$L^*=L^++\epsilon$

3.7 Discovering laws for regular expressions

- There is an infinite variety of laws about regular expressions that might be proposed!
- Is there a general methodology for finding and proving algebraic laws for RE?

  The truth of a law reduces to the equality of two specific languages

  Example: $(L+M)^*=(L^*M^*)^*$

  We can specialize the RE operators to act on symbols, one symbol for each language.

  1. Think of variable L as a symbol a and M as b; we get $(a+b)^*$
  2. Test if $(a+b)^*=(a^*b^*)^*$

- Main theorem

Theorem (3.13). (A lemma that defines $L(E)$ alternatively) Let E be an RE with variables $L_1,\ldots,L_m$. Let C be the corresponding ***concrete*** RE with symbols $a_1,\ldots,a_m$. Then, for any instance languages $S_1,\ldots,S_m$ of $L_1,\ldots,L_m$, ***every string*** w in $L(E)$ can be written $w=w_1\ldots w_k$,

where each w_i is in one of the languages S_{j_i}, and the string $a_{j_1} \dots a_{j_k}$ is in the language $L(C)$. Less formally, we can construct $L(E)$ by starting with each string $a_{j_1} \dots a_{j_k}$ in $L(C)$ and replacing a_{j_i} with a string in S_{j_i}.

The **intuitive meaning** of the theorem is that, starting with a string in the concrete language, you can generate all and only the strings in $L(E)$ by expanding each symbol into a string from the corresponding language associated with that symbol.

Proof. Structural induction on E.

Basis. E is either ϵ, $\emptyset$, or a variable L. If E is L, $L(E) = L$ and the concrete expression C is a, a symbol corresponding to L. Easy.

Induction. If E is the union, concatenation, or closure, we need to prove the claim.

Suppose $E = F + G$. Let C and D be the concrete expressions of F and G. Consider $w \in L(E)$; then w is either in $L(F)$ or $L(G)$. Using the induction hypothesis, w can be obtained by replacing symbols of strings in either $L(C)$ or $L(D)$. Hence, w can be generated from a ***concrete*** string in $L(C + D)$.

Similarly for $E = FG$ and $E = F^*$.

□

- It is a simple method to test a law.

> **Theorem**. $L(E) = L(F)$ ***for all choices of languages*** $L_1, L_2, \dots, L_m$ in E and F if and only if $L(C) = L(D)$ where C and D are the concrete RE's of E and F.

The proof is simple and given in Theorem 3.14.

In the only if direction, use the singleton languages. In the other direction, use Theorem 3.13.

> **Example**. $(L + M)^* = (L^*M^*)^*$ and $L + ML \neq (L + M)L$.

- **Caveat**

  The method may become ***invalid*** if we add other operators (other than union, concatenation, closure)

  Consider $L \cap M \cap N = L \cap M$. It is obviously false, but $\{a\} \cap \{b\} \cap \{c\} = \{a\} \cap \{b\}$.